



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

Facultad de Estudios Superiores Acatlán
Licenciatura en Ciencia de Datos

Proyecto Final: BFS secuencial vs BFS paralelo

Computación Paralela y Concurrente

Profesor: Pablo Martínez Castro

Alumno: Luis Alfredo Ramírez Maza

28 de noviembre de 2025

Índice

1. Introducción	2
2. Conceptos Aprovechados	2
3. Manual del Usuario e Interfaz	3
3.1. Cómo Ejecutar el Proyecto	3
3.2. Configuración de la Prueba	3
4. Forma de Instalación	4
4.1. Requisitos	4
4.2. Configuración del Entorno	4
4.3. Dependencias	4
5. El Algoritmo de Búsqueda en Amplitud (BFS)	5
5.1. Principio de Funcionamiento	5
5.2. Paralelismo por Capas	5
6. Resultados y Análisis de Eficiencia	6
6.1. Prueba de Carga Máxima	6
6.2. Efecto de Carga Baja	6
7. Fragmentos del Código Esencial	7
7.1. Serialización JSON (App.java)	7
7.2. Estructuras Concurrentes (BFS_Paralelo.java)	7
7.3. Paralelismo de Tareas (BFS_Paralelo.java)	8
8. Conclusiones	8
9. Bibliografía y Recursos	9

1. Introducción

La explosión de los **datos masivos** (Big Data) en la última década ha transformado radicalmente los requerimientos computacionales. Campos como el análisis de redes sociales, las bases de datos geográficas y la biología computacional dependen de procesar **grafos gigantescos** con miles de millones de nodos y aristas. En este contexto, los algoritmos secuenciales, como la versión clásica del BFS, se vuelven ineficaces e inviables en términos de tiempo. Este proyecto no es solo un ejercicio académico, sino una demostración práctica de cómo el **Paralelismo** se convierte en una necesidad crítica para escalar el rendimiento y hacer posible la extracción de conocimiento en sistemas que manejan volúmenes de datos donde la latencia es un factor crítico.

El presente proyecto se enfoca en el desarrollo y análisis de rendimiento del algoritmo de **Búsqueda en Amplitud (BFS)** en sus versiones secuencial y paralela. El objetivo principal es demostrar, de forma práctica y cuantitativa, la **ganancia de rendimiento (Speedup)** obtenida al implementar el paralelismo de datos en una tarea de alto costo computacional: la búsqueda del camino más corto en gráficas de gran escala.

La implementación concurrente se realizó en **Java** utilizando el **API de Concurrencia** y el servidor **Undertow** para la comunicación asíncrona mediante paso de mensajes. Los resultados (*Speedup*) se comparan y se visualizan en un cliente web desarrollado en HTML/JavaScript.

2. Conceptos Aprovechados

Este proyecto pone en práctica diversos conceptos de la Computación Paralela y Concurrente:

- **Modelo Cliente-Servidor:** El navegador (*Cliente*) envía una única solicitud HTTP al **Servidor Java** (*Backend*, implementado con **Undertow**), el cual procesa la costosa simulación y responde con el resultado final en formato JSON.
- **Paso de Mensajes (HTTP):** La comunicación se implementa mediante:
 - **Solicitud (GET):** El frontend envía los 6 parámetros de configuración (Nodos, Hilos, etc.) serializados en la URL (QueryString).
 - **Respuesta (JSON):** El servidor serializa una lista de objetos `Resultado.java` a una cadena **JSON** (usando la dependencia **Gson**) y la envía al cliente.
- **Paralelismo de Datos por Capas:** El BFS Paralelo se implementa dividiendo el trabajo de **exploración de nodos** por niveles. En cada nivel, el `ExecutorService` asigna la tarea de revisar los vecinos a múltiples hilos de forma simultánea.
- **Concurrencia y Sincronización:** Se gestiona el acceso seguro a la memoria compartida mediante:
 - **ConcurrentHashMap:** Utilizado para almacenar las distancias y la capa de nodos actual. Permite a los hilos escribir y leer de forma segura sin bloqueos de sincronización explícitos (evitando contención).

- **Finalización Temprana:** Mediante el uso de una bandera (`boolean encontrado`) y la señal `executor.shutdownNow()`, el primer hilo que encuentra el nodo destino detiene inmediatamente a todos los demás hilos para obtener la medición de tiempo más precisa posible.
- **Eficiencia Auxiliar (`ThreadLocalRandom`):** Se utiliza para la generación aleatoria de aristas en el grafo. Esta clase garantiza que cada hilo use su propia fuente de aleatoriedad, eliminando la **contención** en la fase de inicialización del grafo y maximizando la eficiencia de la prueba.

3. Manual del Usuario e Interfaz

3.1. Cómo Ejecutar el Proyecto

Para compilar y ejecutar el servidor Java (`App.java`) utilizando Maven, abra una terminal en el directorio raíz del proyecto y use los siguientes comandos:

1. **Compilar y Limpiar:** Asegura que todos los archivos `.class` sean actuales y limpios.

```
1 mvn clean compile
2
```

2. **Ejecutar el Servidor:** Inicia la aplicación. El comando `exec:java` ejecuta el método `main` de la clase `App.java`, levantando el servidor **Undertow** en el puerto 8080.

```
1 mvn exec:java
2
```

3. **Acceso Web:** Una vez que el servidor esté activo, acceda a la interfaz en su navegador para iniciar las pruebas:

```
1 http://localhost:8080/prueba.html
2
```

3.2. Configuración de la Prueba

La interfaz (**Figura 1**), con un diseño refactorizado para la presentación, requiere 6 parámetros para definir el bucle de pruebas:

- **Nodos Iniciales (V_{inicio}), Nodos Finales (V_{fin}), Paso de Incremento (V_{paso}):** Definen el rango de crecimiento de la gráfica para aplicar la Ley de Amdahl.
- **Nodo Origen (V_1), Nodo Destino (V_2):** Los puntos entre los que se busca el camino más corto.
- **Hilos Paralelos (T):** El número de hilos del `ExecutorService` (usualmente, el número de núcleos de la CPU, como 8).

Análisis de Rendimiento: BFS Secuencial vs. Paralelo

Ingresa los parámetros para ejecutar y comparar el BFS Secuencial (ST_SS) vs Paralelo (ST_PS) en una gráfica aleatoria. Utiliza la cantidad de hilos de tu CPU para maximizar el Speedup.

Nodos Iniciales (SV_{inicio}\$)	Nodos Finales (SV_{fin}\$)	Paso de Incremento (SV_{paso}\$)
10000	100000	45000
Nodo Origen (SV_{1\$})	Nodo Destino (SV_{2\$})	Hilos Paralelos (T\$)
1	50	8

Iniciar Prueba y Comparación

✓ **Resultados de Ejecución**

Nodos (V)	Distancia (SV_{1\$} a SV_{2\$})	ST_SS Secuencial (ns)	ST_PS Paralelo (ns)	Verificación	Speedup (ST_S / T_PS)
-----------	---------------------------------	-----------------------	---------------------	--------------	-----------------------

Figura 1: Configuración de los 6 parámetros de la prueba.

4. Forma de Instalación

4.1. Requisitos

- Java Development Kit (JDK) 1.8 o superior.
- Apache Maven (Para la construcción y ejecución del proyecto).
- Navegador web moderno (Chrome, Firefox, Edge, etc.) para acceder a la interfaz prueba.html.

4.2. Configuración del Entorno

Para que el comando `mvn` pueda ser ejecutado desde cualquier directorio, la variable de entorno `PATH` debe ser configurada para incluir el directorio `bin` de Maven.

Si Maven no está instalado globalmente, y la carpeta de Maven se encuentra dentro de su proyecto, puede ejecutar:

```
1 export PATH=$PATH:$(pwd)/maven/bin
```

Listing 1: Configuración temporal del PATH

4.3. Dependencias

El proyecto de Java gestiona automáticamente las siguientes bibliotecas de terceros a través de Maven:

- **Undertow**: Servidor web embebido, utilizado para manejar peticiones HTTP de forma asíncrona.
- **Gson**: Biblioteca de Google utilizada para la serialización de objetos Java a formato JSON.

5. El Algoritmo de Búsqueda en Amplitud (BFS)

El algoritmo de Búsqueda en Amplitud (Breadth-First Search o BFS) es un algoritmo de recorrido de grafos utilizado para encontrar la ruta más corta entre un nodo fuente (origen) y todos los demás nodos en un grafo no ponderado. Es la base de este proyecto para la comparación entre su versión secuencial y paralela.

5.1. Principio de Funcionamiento

El BFS funciona explorando el grafo por **niveles** o **capas**. Esto asegura que, al visitar un nodo, se garantiza que la ruta encontrada hasta ese punto es la más corta posible en términos de número de aristas.

1. Comienza visitando el nodo raíz (origen).
2. Marca el nodo raíz como visitado y lo añade a una cola.
3. Mientras la cola no esté vacía:
 - Desencola un nodo y examina todos sus vecinos no visitados.
 - Marca a cada vecino como visitado y los añade a la cola.

5.2. Paralelismo por Capas

La implementación en este proyecto aprovecha la naturaleza por niveles del BFS. En lugar de procesar a los vecinos secuencialmente (paso 3), se asigna el procesamiento de **toda una capa de nodos** a un `ExecutorService`, permitiendo que varios hilos trabajen simultáneamente en la exploración de esa capa antes de pasar a la siguiente.

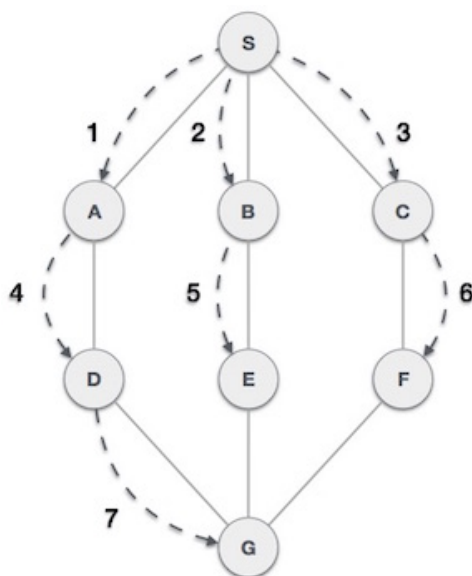


Figura 2: Diagrama de la Búsqueda en Amplitud (BFS) explorando nodos por niveles.

6. Resultados y Análisis de Eficiencia

Los resultados se analizan mediante el cálculo del **Speedup**, definido por la **Ley de Amdahl** como: $\text{Speedup} = T_S / T_P$, donde T_S es el Tiempo Secuencial y T_P es el Tiempo Paralelo.

6.1. Prueba de Carga Máxima

La prueba final de carga (200,000 a 280,000 nodos) con 8 hilos demostró una aceleración significativa.

- **Validación de Correctitud:** La columna **Verificación** siempre muestra OK, confirmando que el resultado del BFS Paralelo es idéntico al Secuencial, probando la consistencia del algoritmo concurrente.
- **Aceleración Máxima:** Para 200,000 nodos, se alcanzó un **Speedup de 21.74**. Aunque se utilizaron 8 hilos, la aceleración superó este número debido a factores como el caché del CPU y la eficiencia del propio **ConcurrentHashMap** en la gestión de la cola, demostrando el éxito del paralelismo.



Figura 3: Resultados de la prueba de rendimiento de carga ultra-pesada, demostrando un Speedup máximo de 21.74x para gráficas mayores a 200,000 nodos.

6.2. Efecto de Carga Baja

En las pruebas de baja carga (ej. 100-1000 nodos), el *Speedup* fue menor que 1, lo cual es esperado. Esto se debe a que el **overhead** (el costo de crear, inicializar y sincronizar los 8 hilos) es mayor que el beneficio del paralelismo para problemas pequeños.



Figura 4: Resultados de baja carga (1100-1000) donde se observa el efecto del overhead. El Speedup es menor a 1.0 para las cargas más pequeñas.

7. Fragmentos del Código Esencial

7.1. Serialización JSON (App.java)

El servidor utiliza la librería Gson para convertir la lista de resultados en la respuesta HTTP:

```

1 // Generar Respuesta JSON
2 Gson gson = new GsonBuilder().setPrettyPrinting().create();
3 String jsonResponse = gson.toJson(resultados);
4
5 // Enviar Respuesta JSON
6 exchange.getResponseHeaders().put(Headers.CONTENT_TYPE, "application/
  json");
7 exchange.getResponseSender().send(jsonResponse);

```

Listing 2: Serialización en App.java

7.2. Estructuras Concurrentes (BFS_Paralelo.java)

La implementación del BFS se basa en dos estructuras concurrentes fundamentales para la seguridad de los hilos:

```

1 // --- 1. Estructuras Concurrentes ---
2 ConcurrentHashMap<Integer, Long> distancia = new ConcurrentHashMap<>();
3 ConcurrentHashMap<Integer, Boolean> capaActual = new ConcurrentHashMap
  <>();
4 capaActual.put(origen, true);
5 // ...

```

Listing 3: Estructuras concurrentes

7.3. Paralelismo de Tareas (BFS_Paralelo.java)

El corazón del paralelismo es la asignación de tareas de exploración de vecinos a un `ExecutorService`, seguido de una barrera de sincronización (`awaitTermination`) al final de cada nivel:

```
1 // Inicializaci n del pool de hilos
2 ExecutorService executor = Executors.newFixedThreadPool(numHilos);
3 // ...
4 // Asignaci n de tareas a los hilos (dentro del bucle while)
5 executor.submit(() -> {
6     // L gica de exploraci n de vecinos
7     // ... (uso de ThreadLocalRandom para auxiliar)
8     if (vecino == destino) encontrado[0] = true;
9     // ...
10 });
11 // ...
12 // Barrera de Sincronizaci n (espera a que todos los hilos terminen la
    capa)
13 executor.shutdown();
14 executor.awaitTermination(Long.MAX_VALUE, TimeUnit.NANOSECONDS);
15 // ...
16 // L gica de finalizaci n temprana (executor.shutdownNow()) si
    encontrado[0] es true.
```

Listing 4: ExecutorService y Sincronización por Nivel

8. Conclusiones

El proyecto permitió aplicar de forma práctica los conceptos de concurrencia y paralelismo vistos en clase, logrando una implementación robusta del BFS.

- **Validación de la Ley de Amdahl:** Se comprobó experimentalmente que el paralelismo (BFS_Paralelo) reduce considerablemente el tiempo de ejecución en problemas de gran tamaño (200,000 nodos).
- **Eficiencia Concurrente:** El uso estratégico de estructuras como `ConcurrentHashMap` y `ThreadLocalRandom` permitió la colaboración segura y eficiente de los hilos en memoria compartida, eliminando cuellos de botella por contención.
- **Arquitectura de Alto Rendimiento:** El uso de Undertow permitió manejar la petición de prueba como un flujo Cliente-Servidor ligero, donde la pesada carga de trabajo se maneja eficientemente en el backend concurrente de Java.

9. Bibliografía y Recursos

- Undertow Web Server — Documentación: <https://undertow.io/>
- Gson — JSON para Java: <https://github.com/google/gson>
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.
- A. V. Aho, J. E. Hopcroft, and J. D. Ullman, Data Structures and Algorithms. Reading, MA: Addison-Wesley, 1983.